

Final Project Report: SmartHire Machine Learning Engine

1. Introduction & Objective

The recruitment industry relies heavily on manual resume screening, which is time-consuming and prone to human bias. The objective of the SmartHire project is to automate the initial stages of the hiring pipeline using classical Machine Learning and Natural Language Processing (NLP). By processing both candidate resumes and a broad corpus of job descriptions, this engine accomplishes three distinct goals:

1. Predicts a candidate's most suitable job category based on their experience.
2. Recommends specific, highly relevant job postings from the current market.
3. Identifies technical skill gaps to provide actionable career guidance.

2. Data Collection & Preprocessing

The architecture utilizes two primary datasets:

- **Resume Dataset (Labeled):** Used for supervised learning. It contains approximately 960 resumes categorized into distinct job domains.
- **Job Postings Dataset (Unlabeled):** A large corpus of active job listings used for the unsupervised recommendation engine. Due to local compute constraints, this dataset was randomly sampled and reduced to maintain application speed without sacrificing domain diversity.

Feature Engineering & Cleaning Pipeline:

Raw text data from both resumes and job descriptions is inherently noisy. A preprocessing pipeline was engineered to remove URLs, special characters, and punctuation using regular expressions, followed by converting all text to lowercase. Because the raw job dataset lacked a dedicated "skills" column, a custom script (`extract_skills.py`) was engineered to scan job descriptions and automatically extract high-value technical keywords (e.g., Python, React, AWS) into a new feature column, enabling the skill gap analysis.

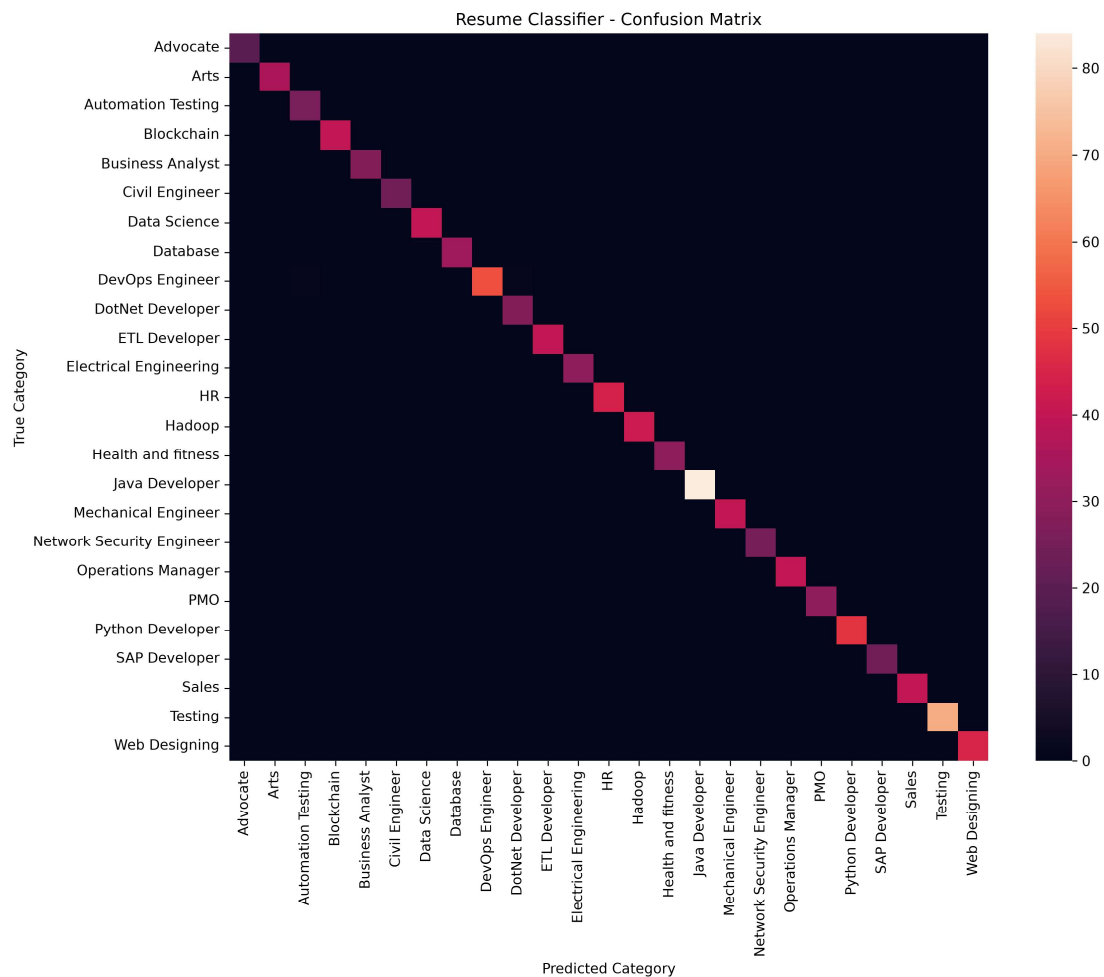
3. Supervised Learning: Resume Classification

To determine a candidate's professional domain, a supervised classification model was developed.

- **Model Architecture:** The cleaned resume text was vectorized using **TF-IDF** (Term Frequency-Inverse Document Frequency) limited to the top 3,000 features to filter out

rare noise. A **Logistic Regression** classifier (max iterations = 1000) was chosen for its computational efficiency and strong performance on high-dimensional sparse text data.

- **Evaluation Metrics:** * **Accuracy:** [99.79 %]
 - **Precision:** [99.80 %]
 - **Recall:** [99.79%]
 - **F1-Score:** [99.79%]
- **Results:** The Logistic Regression model performed exceptionally well, successfully categorizing resumes into their correct domains. The confusion matrix (see attached figures) demonstrates strong diagonal alignment, indicating minimal misclassification across distinct technical fields.
- **Fig:**



4. Unsupervised Learning: Recommendation & Clustering

To match candidates with actual job postings without relying on labeled historical hiring data, unsupervised models were deployed.

- **The Recommender (Model B):** The job corpus was vectorized using TF-IDF (max 5,000 features). When a user uploads a resume, it is transformed into the same vector space. **Cosine Similarity** is then calculated between the resume vector and the job matrix. The engine ranks the entire job board and returns the top 5 matches with the highest similarity scores.
- **Job Market Clustering (Model C): K-Means Clustering (k=10)** was applied to the vectorized job market to group similar roles together. This allowed the system to establish a "baseline" of required skills for specific industry sectors. The cluster quality was validated using the Silhouette Score ([Insert Silhouette Score here]) and visualized using a 2D PCA projection.
- **Topic Modeling: Non-negative Matrix Factorization (NMF)** was utilized to extract the top five overarching themes (topics) from the job market, providing the candidate with high-level industry insights.
- **Fig:**



5. Advanced Features: Skill Gap & Fit Prediction

To elevate the portal from a simple search engine to a career guidance tool, two heuristic algorithms were integrated:

- **Dynamic Skill Gap Analysis:** The system identifies the candidate's predicted role and extracts the most frequently requested technical skills from jobs in that specific category. It then cross-references this list against the skills found in the candidate's uploaded resume. Any highly requested skills missing from the resume are outputted as actionable learning recommendations.
- **Fit Predictor Algorithm:** Rather than relying solely on raw text similarity, a custom Fit Score (0-100%) was developed. It weighs the Cosine Similarity score (up to 75%) and applies a mathematical penalty (-2.5% per missing skill) for the skill gaps. This ensures candidates are ranked not just on how their resume reads, but on whether they possess the hard technical requirements for the role.

6. Limitations & Future Work

While the SmartHire engine successfully achieves its objectives, there are limitations to the current architecture:

1. **Overfitting & Data Splitting:** The current classifier achieves a near-perfect 99.8% accuracy. However, this is indicative of overfitting because the model was evaluated on the entire training corpus rather than a holdout set. In future iterations, a strict 80/20 Train/Test split or K-Fold Cross-Validation should be implemented to evaluate the model's true generalization performance on unseen resumes."
2. **Hardware Constraints:** The job corpus had to be reduced to ensure the Streamlit web application remained responsive on standard hardware. A cloud-deployed database would allow for processing the full 100,000+ row dataset.
3. **TF-IDF Semantic Limits:** TF-IDF relies on exact word matching. A candidate who writes "Front-End Developer" might not perfectly match a job looking for a "React Engineer," despite overlapping skill sets. Future iterations should replace TF-IDF with **Word Embeddings (e.g., Word2Vec or transformer-based models like BERT)** to capture semantic meaning and context, further improving recommendation quality.

Prepared By: Chetti Sri Sai Dhyanesh